

Full-Stack Hardware Realization: Custom 8-Bit Harvard Architecture from VHDL to Physical PCB Integration

Ikramul Hassan Sazid
Department of Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Email: ikramul-hassan.sazid@stud.hshl.de

Muhammad Aamir Ejaz Ejaz
Department of Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Email: muhammad-aamir.ejaz-ejaz@stud.hshl.de

Md Ruhul Kuddus Saleh
Department of Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Email: md-ruhul-kuddus.saleh@stud.hshl.de

Index Terms—Artix-7 FPGA, VHDL, Harvard Architecture, Custom Microprocessor, PCB Design, Digital Logic.

I. INTRODUCTION

In the field of electronics engineering, knowing the silicon architecture which drives computer operations becomes one of the most important fundamentals. While current course syllabi make use of industry-standard microcontrollers, it can be noted that microcontrollers are usually regarded as black boxes in terms of functionality. The hardware engineering task discussed here takes an approach from the bottom up in terms of deconstructing processor functioning by creating a unique 8-bit microprocessor. Creating an 8-bit CPU becomes the best way to learn how to work with Hardware Description Languages (VHDL) and silicon chip designing.

Target Application and Architecture: The target application in this project involves building a highly customized, application-specific processor core. Rather than going for the general-purpose Commercial-Off-The-Shelf (COTS) CPUs, we designed an application-specific 8-bit Harvard Architecture microprocessor that can run the dedicated multiplication and countdown algorithm. The microprocessor will serve as a basic prototype for gaining insight into the ASICs design process. The theoretical logic implemented using VHDL language will finally be physically built on the FPGA device (Artix-7), which in turn will be assembled on our own PCB board.

II. CONCEPT DESCRIPTION

System Overview and Requirements: The theoretical basis of this customized microprocessor is based on a well-defined set of hardware requirements and architectural constraints modeled using the Systems Modeling Language (SysML). As seen in Fig. 1, the major requirement is CPU-100, which defines that the 8-bit processor core should be built. To meet this requirement, the architecture has some mandates: the use of Harvard Architecture to separate the instruction

bus and the data bus, 8-bit continuous data path, and 4-bit addressing which reduces the memory depth to 16 addresses in each bus.

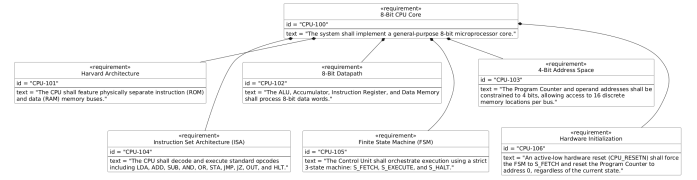


Fig. 1: SysML Requirement Diagram defining the core architectural constraints of the CPU.

Control Flow and Dynamic Behavior: Control Unit FSM coordinates the dynamic behavior of the CPU. Fig. 6 shows the SysML Activity Diagram for the execution sequence of hardware logic. On active low hardware reset signal, the value of Program Counter (PC) becomes zero, and the CPU starts the infinite deterministic loop with Fetch and Execute stages.

During the S_FETCH phase, the instruction is loaded from the ROM into the Instruction Register (IR), and the PC is incremented. The $S_EXECUTE$ phase decodes the upper nibble of the IR (bits 7..4) to route the correct control signals to the datapath. For example, arithmetic opcodes like ADD and SUB route memory data into the Arithmetic Logic Unit (ALU), while memory-access opcodes like STA route the Accumulator's state back into the RAM. This cycle repeats endlessly until the HLT (Halt) opcode forces the control logic into a frozen state, protecting the final calculated values.

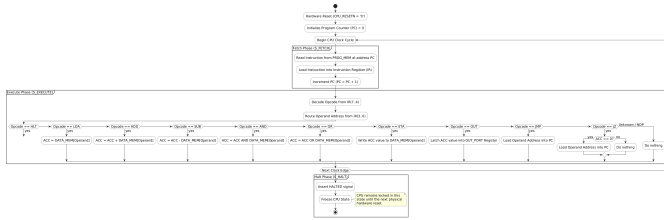


Fig. 2: SysML Activity Diagram detailing the Fetch-Execute cycle and opcode branching logic.

Component Interaction and Timing: To physically realize the activity flow in silicon, precise clock-cycle synchronization is required across the datapath. Fig. 3 presents a SysML Sequence Diagram illustrating the hardware messaging between the synthesized VHDL entities over two distinct clock cycles.

During Clock Cycle 1 (Fetch), the Control Unit asserts the PC_INC signal, prompting the ROM to output the 8-bit instruction word to the IR. During Clock Cycle 2 (Execute), the decoded operation dictates which hardware lifelines become active. If an ADD instruction is decoded, the IR routes the 4-bit operand address to the RAM, which subsequently feeds the 8-bit data word to the ALU's B-input. Concurrently, the Control Unit pulses the ALU_OP vector and the ACC_LOAD signal to safely latch the resulting sum. Alternatively, a control-flow instruction such as JMP (Jump) bypasses the ALU entirely, asserting PC_LOAD to overwrite the Program Counter with a new target address.

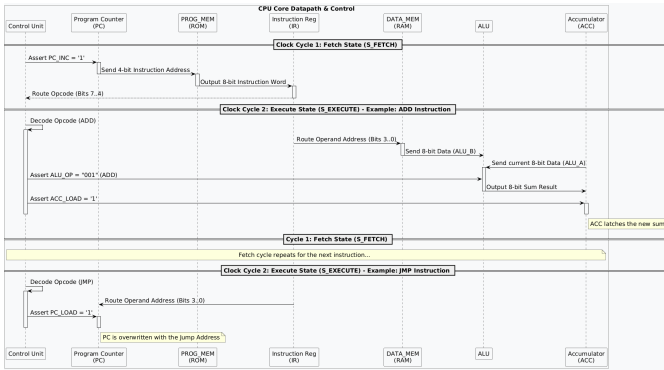


Fig. 3: SysML Sequence Diagram illustrating the clock-by-clock data routing and hardware messaging.

Fig. 4: High-Level Block Diagram of the 8-Bit CPU Architecture.

Devices, Sensors, and Actuators utilized:

- **Central Processing:** Xilinx Artix-7 100T FPGA (executing the VHDL custom logic).
- **Sensors (Input):** 16-bit physical DIP switches, simulating raw external data feeds (acting as the Multiplicand and Multiplier).

- **Actuators (Output):** An 8-digit, time-multiplexed 7-Segment Display driven at 760 Hz, providing instantaneous visual telemetry of the CPU's accumulator state.

III. PROJECT/TEAM MANAGEMENT

Methods and Task Breakdown: An iterative approach of hardware/software design was adopted for the project. The process was divided into three main stages that include RTL Design (writing and testing the VHDL), Software Integration (writing 16-line assembly program that will be placed in the ROM) and Physical Implementation (Altium Designer PCB routing). All tasks were managed by the use of version control where all VHDL blocks were tested in testbenches. **Team Member Roles:**

- **Ikramul Hassan Sazid:** Architected the core datapath and RTL logic in VHDL. Managed the memory modules (prog_mem.vhd, data_mem.vhd), implemented the memory-mapped I/O, and Authored the systems engineering documentation (SysML)
- **Muhammad Aamir Ejaz Ejaz:** Developed the Control Unit Finite State Machine (control_unit.vhd). Handled the instruction decoding logic, clock division algorithms, and integrated the Binary-Coded Decimal (BCD) converter for base-10 display outputs.
- **Md Ruhul Kuddus Saleh:** Designed the custom 16-line assembly software to execute the multiplication and self-halting countdown loops, and verified the unified architecture via top-level testbenches. Integrated two showcase codes together. Handled PCB development and realization.

IV. TECHNOLOGIES

The technological approach spans bare-metal coding to physical board fabrication:

- **Programming Languages:** VHDL (VHSIC Hardware Description Language) was exclusively used to model the digital logic. A custom **Assembly Language** was drafted specifically for the processor's Instruction Set Architecture (ISA).
- **Synthesis and Routing:** Xilinx Vivado was utilized for RTL synthesis, mapping, and bitstream generation targeting the Artix-7 architecture.
- **PCB Design:** Altium Designer was employed for schematic capture, footprint generation, and physical copper routing of the final integrated board.
- **Data Interfaces:** A custom memory-mapped I/O protocol was developed, hijacking memory addresses 14 and 15 to feed raw physical switch data directly into the Arithmetic Logic Unit (ALU).

V. IMPLEMENTATION

A. Architecture and Integrated VHDL Modules

The static structure and processing core of the system are centrally integrated within the `cpu_core.vhd` entity. This top-level module acts as the primary datapath and control nexus, instantiating all essential computational and memory

components. A significant engineering refinement for this project was the transition from a standard Von Neumann architecture (which used a single unified memory) to a modern Harvard architecture. This change was necessitated by the need to efficiently manage two distinct software requirements concurrently: the specialized multiplication software loop and the large volume of temporary variable data (sum, multiplicand, multiplier). The previous single-memory approach proved inadequate due to conflicting access limitations. The expanded Harvard design, featuring physically separate instruction and data buses, optimizes performance and solves this contention by enabling simultaneous access.

The key VHDL modules instantiated within the `cpu_core.vhd` and their specific roles include:

- **pc_4bit.vhd (Program Counter):** Generates the specific 4-bit address used to index into the instruction memory. The deliberately constrained 16-byte address space is an intentional architectural decision for this highly specialized computer. It also handles the address increment logic.
- **prog_mem.vhd (Instruction ROM):** The pre-loaded instruction memory containing the application-specific assembly program. It receives the 4-bit address from the program counter and outputs the corresponding 8-bit instruction word.
- **control_unit.vhd (Control Unit):** Manages the system's central finite state machine (FSM) and instruction decoding. It generates all internal control signals (memory write enable, ALU operation code, register load, etc.) and orchestrates the dynamic behavior of the datapath.
- **data_mem.vhd (Data RAM):** Stores variable data used during computation. The Harvard design enables the control unit to index into this memory using the 4-bit operand address simultaneously while fetching the next instruction from ROM. It incorporates memory-mapped I/O (intercepting reads to specific addresses and rerouting them to live switch data) and an asynchronous reset to physically clear temporary variables during a hardware reset.
- **alu_8bit.vhd (ALU):** Performs all mathematical and logical operations (ADD, SUB, AND, OR) on standard 8-bit data words as commanded by the control unit.
- **register_8bit.vhd (Accumulator):** The primary 8-bit data register holding temporary ALU calculation results and providing data for logical operations and memory writes.

B. Dynamic Control FSM Logic

The system's operation is precisely governed by the structured finite state machine within `control_unit.vhd`. This strict 3-state architecture is deterministic and specialized for the pre-loaded application, intentionally sacrificing general-purpose principles for hardware-level optimizations.

- 1) **S_FETCH:** The FSM begins in this state. The program counter generates the instruction address, which

is routed to the instruction ROM. The returned 8-bit instruction word is latched into the Instruction Register (IR) by the datapath. Simultaneously, the program counter increments its address in preparation for the next cycle.

- 2) **S_EXECUTE:** This is the core operational state. The control unit decodes the 4-bit opcode located in the upper nibble of the IR. Based on the unique instruction, it generates specific control signals to all necessary VHDL modules. The 4-bit operand address is used to access data variables in the RAM or provide immediate data for logical operations.

Key to the unique multiplication and countdown functionality are several application-specific instructions and hardware optimizations managed during this state:

- Standard instructions like Load (LDA), Store (STA), Jump (JMP), and Jump-if-Zero (JZ) execute the software's multiplication loop via repeated addition.
- The system utilizes "self-cleaning memory." Instead of a complex software loop to clear variables, the customized reset feature physically zeroes out temporary results upon a hardware reset, linking hardware function directly to application needs.
- Memory-mapped I/O reads (addresses 14 and 15) bypass the RAM array and route directly to the live switch data inputs.
- Special custom instructions like CDWN trap the control unit FSM within its own specialized, infinite hardware execution loop. This physically prevents the CPU from fetching new instructions while it independently counts down the accumulator and monitors its own termination condition (a zero count).

- 3) **S_HALT:** This permanent state is triggered when the control unit decodes the HLT opcode (or its equivalent from the custom countdown loop termination). All internal clock cycles within the core are suspended, physically freezing the CPU. The CPU remains in this state, holding the final visual result on the human-observable display until a physical hardware reset occurs, restoring the FSM and cleansing the temporary variables.

Use Cases:

- **Hardware Multiplication:** The CPU loads variables from physical switches and executes a software loop (LDA, ADD, SUB, JZ, JMP) to perform mathematical multiplication via repeated addition.
- **Visual Countdown:** Upon completing the calculation, the software transitions to a countdown loop, outputting the decremented accumulator state to the `seg7_controller` until it reaches zero.
- **Asynchronous Memory Scrubbing:** When the user presses the CPU_RESETN button, the custom data memory asynchronously wipes the specific RAM addresses holding the calculation variables, ensuring a clean state for the next use case without relying on software initial-

ization.

VI. PCB REALIZATION

We developed the PCB to hold the artix 7 fpga

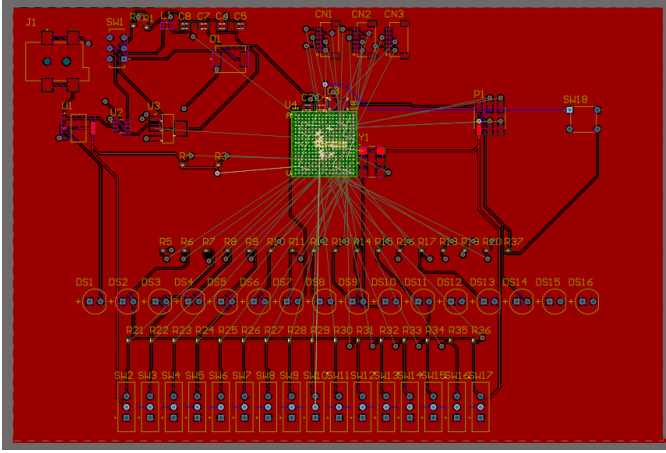


Fig. 5: PCB for artix 7 FPGA

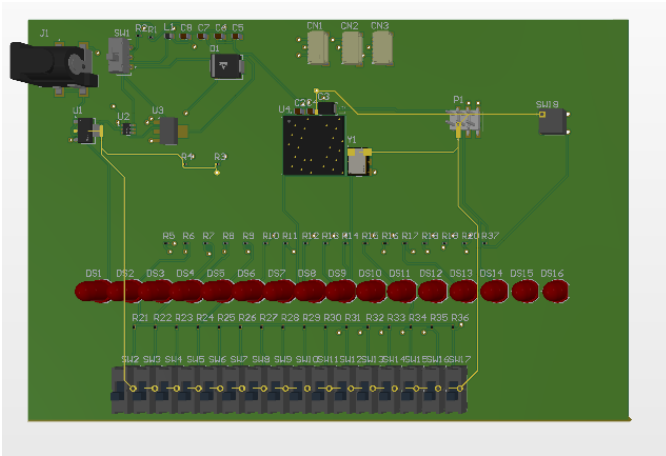


Fig. 6: PCB for artix 7 FPGA

VII. RESULTS

The system was successfully synthesized and tested on the Artix-7 hardware.

Base-10 Translation: Initially, the CPU's raw 8-bit output rendered in hexadecimal. To make the interface human-readable, a dedicated Binary-to-BCD (`bin_to_bcd.vhd`) module was integrated into the top-level architecture. This successfully transformed the raw binary ALU results into distinct Hundreds, Tens, and Ones decimal chunks, routing them flawlessly to the 7-segment display.

Execution and Clock Management: The custom 16-line assembly program executed perfectly. By manipulating the clock divider inside `top_nexys_a7.vhd`, the CPU was calibrated to run at approximately 24 Hz. At this frequency, observers can physically watch the 7-segment display pause while the processor "thinks" through the multiplication loop

(e.g., $5 \times 4 = 20$), followed immediately by a visible, real-time countdown from 20 to 0 before the software deliberately triggers the `S_HALT` state.

Fig. 7: Successful Output of the 8-Bit CPU on the Hardware Display.

VIII. SOURCES/REFERENCES

REFERENCES

- [1] Project Repository: Collaboration logs, VHDL source files, and Altium schematics are documented in the team GitHub repository. Project Github repository: https://github.com/IkramulHassanSazid/SS2026_HWE_Lab_B3
- [2] Digilent, "Nexys A7 FPGA Board Reference Manual," 2023.
- [3] Xilinx, "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2022.